

Chapter 3: Traffic Analysis

Lecture by Assist. Prof. Dr. Chanankorn Jandaeng

Walailak University

Duration: 90 minutes

Goal: Understand normal network traffic behavior and learn to use Wireshark to analyze, filter, and interpret packet data — preparing for anomaly and attack detection in the next chapter.

Lecture Overview

1. Introduction to Network Traffic
2. Wireshark Interface
3. Packet Dissection
4. Filters
5. Normal Traffic Behavior
6. Protocol Patterns
7. Tshark (CLI Automation)
8. PyShark (Python Integration for ML & Research)

1. Introduction to Network Traffic

Definition:

Network traffic is the flow of data packets across a network between devices.

Key Characteristics:

- Direction: inbound / outbound
- Type: unicast, broadcast, multicast
- Volume and timing vary by use

Purpose of Analysis:

- Understand communication structure
- Identify anomalies
- Support troubleshooting and security investigations

Why Analyze Normal Traffic First?

“You can’t detect abnormal if you don’t understand normal.”

- Establishes baseline patterns
- Reveals expected protocols and services
- Allows performance tuning and policy validation
- Forms the **foundation for intrusion detection**

2. Wireshark Overview

Wireshark:

An open-source network protocol analyzer used to capture and inspect packets in real time.

Core Capabilities:

- Live capture and offline analysis
- Deep packet inspection (Layer 2–7)
- Filtering, decoding, and exporting data

Common Use Cases:

- Network troubleshooting
- Performance monitoring
- Security auditing

Wireshark Interface Components

Section	Description
Menu / Toolbar	Access to capture, analysis, and view settings
Packet List Pane	Summary of all captured packets
Packet Details Pane	Layer-by-layer protocol dissection
Packet Bytes Pane	Raw hexadecimal and ASCII representation

Tip:

Students should familiarize themselves with pane hierarchy: **Frame → Ethernet → IP → TCP/UDP → Application.**

Capture Basics

Steps to Capture:

1. Select an active network interface.
2. Start capture (Ctrl+E).
3. Observe live packet flow.
4. Stop capture and save the `.pcap` file.

Best Practice:

Always use **filter expressions** to minimize irrelevant traffic during long captures.

3. Packet Dissection

Each captured packet is composed of multiple protocol layers.

Example Structure:

```
Frame 35: 74 bytes on wire (592 bits)
Ethernet II: Src=00:1A:2B:3C:4D:5E, Dst=08:00:27:AB:CD:EF
Internet Protocol Version 4: Src=192.168.1.10, Dst=8.8.8.8
Transmission Control Protocol: Src Port=51234, Dst Port=80
Hypertext Transfer Protocol
```


3. Packet Dissection

Layer Analysis:

1. **Ethernet (L2)** – MAC addressing
2. **IP (L3)** – Routing and addressing
3. **TCP/UDP (L4)** – Transport control
4. **Application (L7)** – HTTP, DNS, etc.

Understanding Frame, Packet, and Segment

Term	Layer	Description
Frame	Data Link	Encapsulates MAC addresses
Packet	Network	Includes IP header for routing
Segment	Transport	Contains TCP or UDP data

Visualization:

Ethernet Frame → IP Packet → TCP Segment → Application Data

4. Filter Expressions

Wireshark supports two major types of filters:

1. **Capture Filters** — limit what gets captured
2. **Display Filters** — show or hide packets in the GUI

Capture Filters

Syntax: Based on `libpcap` rules.

Filter	Description
<code>host 8.8.8.8</code>	Capture only traffic to/from 8.8.8.8
<code>port 80</code>	Capture only HTTP traffic
<code>net 192.168.1.0/24</code>	Capture local subnet
<code>tcp</code>	Capture only TCP traffic

Use Case:

Efficient for long captures or limited storage.

Display Filters

Syntax: Wireshark's internal filter language.

Filter	Description
<code>ip.addr == 192.168.1.10</code>	Show packets from or to IP
<code>http.request</code>	Show only HTTP requests
<code>dns && udp</code>	Show DNS queries over UDP
<code>tcp.flags.syn == 1</code>	Show SYN packets (start of TCP handshake)
<code>frame.len > 500</code>	Show large packets only

Tip:

Combine with logical operators:

```
ip.src == 10.0.0.1 && tcp.port == 80
```

5. Understanding Normal Network Behavior

Baseline Observation:

- Traffic usually dominated by DNS, ARP, HTTP(S), and ICMP.
- TCP sessions show clear **SYN → SYN/ACK → ACK** sequence.
- ARP requests appear frequently in LAN environments.
- DNS lookups precede web access.
- ICMP echoes (ping) occur occasionally for diagnostics.

Normal Behavior Example: Web Browsing Session

Sequence:

1. DNS Query → DNS Response
2. TCP Handshake (Client ↔ Web Server)
3. HTTP Request → HTTP Response
4. TCP Termination

Wireshark View:

1. DNS Standard query A [www.example.com] (http://www.example.com)
2. DNS Standard query response 93.184.216.34
3. TCP SYN, SYN/ACK, ACK (3-way handshake)
4. HTTP GET /index.html
5. HTTP 200 OK (text/html)

Normal DNS Traffic Pattern

- Usually **UDP port 53**
- Query/response pairs with short intervals
- Small payloads (<512 bytes)
- Frequent in networks with web activity

Display Filter:

`dns.flags.response == 0` → queries only

`dns.flags.response == 1` → responses only

Normal ARP Activity

- Local broadcast (MAC ff:ff:ff:ff:ff:ff)
- Regular mapping queries
- Common in startup or idle phases

Example:

```
Who has 192.168.1.1? Tell 192.168.1.10
```

Display Filter:

```
arp
```

Insight:

ARP should be consistent and low-volume — spikes can indicate scanning or spoofing, which we'll study later.

Normal ICMP Behavior

- Ping requests/replies (Echo / Echo Reply)
- Used for connectivity testing
- Occasional in normal traffic, but not constant

Display Filter:

`icmp.type == 8` → Echo request

`icmp.type == 0` → Echo reply

Normal TCP Session Characteristics

- Proper **SYN** → **SYN/ACK** → **ACK** handshake
- Balanced request–response packets
- Graceful closure (**FIN** / **ACK**)
- Minimal retransmissions

Display Filter Example:

`tcp.analysis.retransmission` → Detects anomalies, usually absent in normal conditions.

6. Common Protocol Traffic Patterns

Protocol	Typical Port	Behavior	Normal Indicators
HTTP	80	Request/response	Text data, short transactions
HTTPS	443	Encrypted	TLS handshake visible, encrypted payload
DNS	53 (UDP)	Query/response	Pairs of small packets
ARP	N/A	LAN resolution	Regular broadcast, no flooding
ICMP	N/A	Connectivity check	Low frequency, balanced replies
DHCP	67/68	Address allocation	Short sequences during startup

7. Tshark — Command-Line Packet Analysis

What is Tshark?

Tshark is the **command-line version of Wireshark**, offering the same packet capture and analysis features without a graphical interface.

Key Advantages:

- Lightweight and fast
- Ideal for servers and headless systems
- Supports scripting and automation
- Integrates well with **log collection** and **SIEM tools**

Use Case:

When GUI access is unavailable or batch analysis is required.

Tshark Installation

Tshark is included with Wireshark by default.

To check installation:

```
tshark -v
```

Example Output:

```
TShark (Wireshark) 4.0.3 (Git v4.0.3)
```

If not installed:

- **Linux (Debian/Ubuntu):** `sudo apt install tshark`
- **macOS (Homebrew):** `brew install wireshark`
- **Windows:** Comes with Wireshark package.

Capturing Live Traffic

Basic Command:

```
sudo tshark -i eth0
```

Parameters:

- `-i` : Specify interface (e.g., eth0, wlan0)
- `Ctrl + C` : Stop capture
- By default, Tshark shows live summary lines for each packet.

Example Output:

```
1    0.000000 192.168.1.10 → 8.8.8.8    DNS    Standard query A www.google.com
2    0.020000 8.8.8.8 → 192.168.1.10  DNS    Standard query response
```

Saving Capture Files

To save packets into a `.pcap` file for later Wireshark review:

```
sudo tshark -i eth0 -w capture.pcap
```

View capture summary:

```
tshark -r capture.pcap
```

Tips:

- Use meaningful filenames (include timestamp).
- Store captures in a secure folder — packet data may contain sensitive info.

Applying Capture Filters

Tshark supports **Berkeley Packet Filter (BPF)** syntax (same as Wireshark's capture filters).

Examples:

```
sudo tshark -i wlan0 port 80
sudo tshark -i eth0 host 192.168.1.5
sudo tshark -i eth0 tcp or udp
```

Explanation:

- `port 80` → capture only HTTP
- `host 192.168.1.5` → capture traffic to/from this host
- `tcp or udp` → capture both protocols

Applying Display Filters

Tshark can also use **Wireshark-style display filters** during output.

Example:

```
tshark -r capture.pcap -Y "http.request"
```

Additional Examples:

```
tshark -r capture.pcap -Y "dns.flags.response == 1"  
tshark -r capture.pcap -Y "ip.src == 10.0.0.5 && tcp.port == 443"
```

Purpose:

Quickly extract relevant traffic without loading Wireshark GUI.

Extracting Specific Fields

Tshark allows field-level extraction — ideal for scripting and data analysis.

Syntax:

```
tshark -r capture.pcap -T fields -e frame.number -e ip.src -e ip.dst -e tcp.port
```

Extracting Specific Fields

Sample Output:

```
1 192.168.1.10 142.250.190.14 443
2 192.168.1.10 8.8.8.8 53
```

Useful Options:

- `-T fields` → Output specific columns
- `-e` → Select fields
- `-E header=y` → Include column headers
- `-E separator=,` → CSV output for analysis

Filtering by Time or Packet Count

Capture for a duration (e.g., 30 seconds):

```
sudo tshark -a duration:30 -i eth0 -w short_capture.pcap
```

Limit number of packets:

```
sudo tshark -c 100 -i eth0
```

Explanation:

- `-a duration:n` stops capture after n seconds.
- `-c n` stops after n packets captured.

Summarizing Capture Statistics

Tshark can generate simple summaries and counts for quick inspection.

Top Talkers:

```
tshark -r capture.pcap -q -z conv,ip
```

Summarizing Capture Statistics

Protocol Hierarchy:

```
tshark -r capture.pcap -q -z io,stat,1
```

Output Example:

```
Protocol Hierarchy Statistics
Ethernet
  IPv4
    TCP
      HTTP
    UDP
      DNS
```

Insight:

These summaries provide a quick overview of normal traffic composition.

Using Tshark for Baseline Analysis

Goal:

Identify typical traffic patterns and their frequencies.

Example Workflow:

1. Capture 2 minutes of normal activity
2. Filter DNS, HTTP, and HTTPS traffic
3. Export to CSV
4. Compute counts per protocol

Using Tshark for Baseline Analysis

Command Example:

```
tshark -r normal.pcap -T fields -e _ws.col.Protocol | sort | uniq -c
```

Result:

```
50 DNS  
120 HTTP  
300 HTTPS  
20 ICMP
```

Interpretation:

Establishes a **normal protocol frequency baseline** for later anomaly detection.

Automating Tshark for Daily Logs

Example Bash Script:

```
#!/bin/bash
DATE=$(date +"%Y%m%d_%H%M%S")
tshark -i eth0 -a duration:60 -w /var/log/captures/traffic_${DATE}.pcap
```

Usage:

- Run automatically via cron every hour.
- Produces rolling captures for long-term trend analysis.

Comparing Tshark and Wireshark

Feature	Wireshark	Tshark
Interface	Graphical	Command-line
Ease of Use	Intuitive GUI	Requires commands
Automation	Limited	Excellent
Performance	Heavier	Lightweight
Preferred Use	Local analysis	Remote / batch analysis

Key Takeaways on Tshark

- Tshark = Wireshark CLI equivalent
- Ideal for **remote capture, scripting, and automation**
- Filters, fields, and export options mirror Wireshark's logic
- Perfect for building **datasets for traffic anomaly detection** (next lecture)

8. PyShark — Automating Traffic Analysis with Python

What is PyShark?

PyShark is a **Python wrapper for Tshark**, enabling programmatic packet capture, filtering, and parsing.

Purpose:

- Automate network analysis
- Extract structured data from `.pcap` files
- Integrate traffic data with **machine learning workflows**

Core Advantage:

Brings Wireshark/Tshark power into **Python scripts** for data-driven cybersecurity research.

Installing PyShark

Command:

```
pip install pyshark
```

Prerequisite:

Wireshark/Tshark must be installed and accessible in the system PATH.

Verification:

```
python -m pyshark --version
```

Basic Capture Example

Live Capture:

```
import pyshark

capture = pyshark.LiveCapture(interface='eth0')
capture.sniff(timeout=10)

for packet in capture:
    print(packet.highest_layer, packet.ip.src, "→", packet.ip.dst)
```

Explanation:

- Captures 10 seconds of live traffic
- Displays each packet's top layer and IP pair
- Useful for **quick network behavior snapshots**

Reading from a PCAP File

Offline Analysis Example:

```
import pyshark

cap = pyshark.FileCapture('normal_traffic.pcap')

for pkt in cap:
    if 'IP' in pkt:
        print(pkt.ip.src, '→', pkt.ip.dst, pkt.highest_layer)
```

Use Case:

Ideal for research labs — analyzing large datasets without GUI tools.

Applying Display Filters in PyShark

Example:

```
cap = pyshark.FileCapture('network.pcap', display_filter='http.request')  
  
for pkt in cap:  
    print(pkt.http.host, pkt.http.request_method, pkt.http.request_uri)
```

Explanation:

- Uses Wireshark-style filter syntax
- Filters only HTTP request packets
- Extracts useful metadata for traffic profiling

Exporting Structured Data for Analysis

```
import pandas as pd
import pyshark

cap = pyshark.FileCapture('normal_traffic.pcap', display_filter='ip')

data = []
for pkt in cap:
    data.append({
        'time': pkt.sniff_time,
        'src': pkt.ip.src,
        'dst': pkt.ip.dst,
        'protocol': pkt.highest_layer,
        'length': pkt.length
    })
```

cont...

Exporting Structured Data for Analysis

```
df = pd.DataFrame(data)
df.to_csv('normal_traffic.csv', index=False)
print(df.head())
```

Outcome:

Generates a **CSV dataset** for visualization or machine learning preprocessing.

Extracting Statistical Features

Objective: Build baseline metrics for ML model training.

Example:

```
import statistics
packet_lengths = [int(pkt.length) for pkt in cap]
avg_len = statistics.mean(packet_lengths)
print("Average packet length:", avg_len)
```

Applications:

- Feature engineering (packet size, flow duration, protocol frequency)
- Input for ML-based anomaly detection

Live Filtered Capture and Analysis

Example:

```
capture = pyshark.LiveCapture(interface='wlan0', display_filter='dns')  
  
for pkt in capture.sniff_continuously(packet_count=10):  
    print(f"[{pkt.sniff_time}] DNS Query: {pkt.dns.qry_name}")
```

Usage:

- Observe **real-time DNS queries**
- Demonstrates how normal DNS traffic behaves under live observation

Combining PyShark and Tshark Commands

Hybrid Workflow Example:

```
import subprocess
import pyshark

# Step 1: Run Tshark for short capture
subprocess.run(['tshark', '-i', 'eth0', '-a', 'duration:30', '-w', 'short_capture.pcap'])

# Step 2: Analyze captured data with PyShark
cap = pyshark.FileCapture('short_capture.pcap', display_filter='tcp')
print("Total TCP packets:", len(list(cap)))
```

Purpose:

Demonstrates how Tshark and PyShark integrate for automated **data pipeline development** in cybersecurity research.

Benefits of Using PyShark

Feature	Benefit
Automation	Enables scheduled and continuous packet analysis
Integration	Works with Pandas, NumPy, and ML libraries
Reproducibility	Allows repeatable experiments for dataset creation
Scalability	Ideal for large-scale or distributed traffic analysis systems

PyShark in Research Context

Applications in Academia:

- Traffic dataset generation for supervised learning
- Network behavior modeling
- Feature extraction for intrusion detection
- Protocol pattern classification

Key Takeaways on PyShark

- Python-based interface for Tshark packet analysis
- Enables automation and data extraction for research
- Bridges network monitoring with machine learning
- Foundational tool for advanced cybersecurity analytics

End of Lecture

Assist. Prof. Dr. Chanankorn Jandaeng

Walailak University

Contact: chatchanan.ja@mail.wu.ac.th